커맨드 레퍼런스 (38개)

요약

그룹	수	CRUD	설명
Actor	6	CRUD	레벨 내 액터 생성/조회/수정/삭제
ActorComponent	3	CRD	레벨 액터의 컴포넌트 추가/조회/제거
Blueprint	7	CRUD	블루프린트 에셋 생성/조회/컴파일/삭제/스폰
BlueprintComponent	3	CRD	블루프린트 SCS 컴포넌트 추가/조회/제거
Property	2	U	블루프린트 컴포넌트 속성 설정
Material	6	RU	머티리얼 조회/적용/색상 변경
StaticMesh	2	R	스태틱 메시 에셋 조회
BlueprintVariable	2	CD	블루프린트 변수 추가/제거
BlueprintGraph	7	CRUD	블루프린트 이벤트 그래프 노드/연결 조작

Actor (6개)

CRUD — 레벨 내 액터 조작

커맨드	CRUD	설명
`get_actors_in_level`	R	레벨의 모든 액터 목록 조회
`find_actors_by_name`	R	이름으로 액터 검색 (부분 일치)
`spawn_actor`	C	새 액터를 레벨에 스폰

커맨드	CRUD	설명
`delete_actor`	D	레벨에서 액터 삭제
`set_actor_transform`	U	액터의 위치/회전/스케일 변경
`read_actor_content`	R	액터의 상세 정보 조회

`get_actors_in_level`

레벨에 있는 모든 액터를 반환한다.

- 파라미터: 없음
- 응답: `{ count, actors: [{ name, class, location, rotation, scale }] }`

`find_actors_by_name`

이름의 일부로 액터를 검색한다.

- 파라미터:
- `search_text` (str, 필수) — 검색할 텍스트 (대소문자 구분)
- 응답: `{ count, actors: [{ name, class, location, rotation, scale }] }`

`spawn_actor`

지정한 클래스의 액터를 레벨에 스폰한다.

- 파라미터:
- `type` (str, 필수) — 액터 클래스명 (예: `"StaticMeshActor"`, `"PointLight"`, `"CameraActor"`)
- `name` (str, 필수) — 고유한 액터 이름
- `location` (list[float], 선택) — `[x, y, z]` 월드 좌표 (기본: `[0, 0, 0]`)
- `rotation` (list[float], 선택) — `[pitch, yaw, roll]` 도 단위 (기본: `[0, 0, 0]`)
- `scale` (list[float], 선택) — `[x, y, z]` 스케일 (기본: `[1, 1, 1]`)
- 응답: `{ status: "success" }`

`delete_actor`

레벨에서 액터를 삭제한다.

- 파라미터:
- `name` (str, 필수) — 삭제할 액터의 정확한 이름

- 응답: `{ status: "success" }`

`set_actor_transform`

액터의 트랜스폼(위치/회전/스케일)을 변경한다. 제공한 값만 변경되고 나머지는 유지된다.

- 파라미터:
- `name` (str, 필수) — 액터 이름
- `location` (list[float], 선택) — `[x, y, z]`
- `rotation` (list[float], 선택) — `[pitch, yaw, roll]`
- `scale` (list[float], 선택) — `[x, y, z]`
- 응답: `{ status: "success" }`

`read_actor_content`

액터의 이름, 클래스, 트랜스폼 정보를 반환한다.

- 파라미터:
- `name` (str, 필수) — 액터 이름
- 응답: `{ name, class, location, rotation, scale }`

Actor Component (3개)

CRD — 레벨 액터의 컴포넌트 조작 (Update는 Property 커맨드로 수행)

커맨드	CRUD	설명
`add_component_to_actor`	C	액터에 컴포넌트 추가
`read_components_from_actor`	R	액터의 컴포넌트 목록 조회
`remove_component_from_actor`	D	액터에서 컴포넌트 제거

`add_component_to_actor`

레벨 내 액터에 새 컴포넌트를 추가한다.

- 파라미터:
- `name` (str, 필수) — 대상 액터 이름

- `component_class` (str, 필수) — 컴포넌트 클래스명 (예: `"StaticMeshComponent"`, `"PointLightComponent"`)
- `component_name` (str, 선택) — 컴포넌트 이름 (생략 시 자동 생성)
- 응답: `{ status: "success", component_class, component_name }`

`read_components_from_actor`

액터에 붙어있는 모든 컴포넌트를 반환한다.

- 파라미터:
- `name` (str, 필수) — 액터 이름
- 응답: `{ actor_name, count, components: [{ name, class, is_root, relative_location, ... }] }`

`remove_component_from_actor`

액터에서 컴포넌트를 제거한다. 루트 컴포넌트는 제거할 수 없다.

- 파라미터:
- `name` (str, 필수) — 대상 액터 이름
- `component_name` (str, 필수) — 제거할 컴포넌트 이름
- 응답: `{ status: "success" }`

Blueprint (7개)

CRUD — 블루프린트 에셋 관리 + 레벨 스폰

커맨드	CRUD	설명
`get_blueprints`	R	프로젝트의 블루프린트 목록 조회
`find_blueprints_by_name`	R	이름으로 블루프린트 검색
`create_blueprint`	C	새 블루프린트 생성
`read_blueprint_content`	R	블루프린트 내용 조회
`compile_blueprint`	U	블루프린트 컴파일
`delete_blueprint`	D	블루프린트 삭제
`spawn_blueprint_actor`	C	블루프린트를 레벨에 스폰

`get_blueprints`

프로젝트 내 모든 블루프린트 에셋을 반환한다.

- 파라미터:
- `search_path` (str, 선택) — 검색 경로 (기본: `"/Game/"`)
- 응답: `{ count, blueprints: [{ name, path, package_path }] }`

`find_blueprints_by_name`

이름의 일부로 블루프린트를 검색한다.

- 파라미터:
- `search_text` (str, 필수) — 검색할 텍스트 (대소문자 구분)
- 응답: `{ count, blueprints: [{ name, path, package_path }] }`

`create_blueprint`

새 블루프린트 에셋을 `/Game/Blueprints/` 경로에 생성한다.

- 파라미터:
- `name` (str, 필수) — 블루프린트 이름
- `parent_class` (str, 선택) — 부모 클래스명 (기본: `"Actor"`)
- 응답: `{ name, path }`

`read_blueprint_content`

블루프린트의 변수, 함수, 이벤트 그래프, 인터페이스 정보를 반환한다.

- 파라미터:
- `blueprint_path` (str, 필수) — 에셋 경로 (예: `"/Game/Blueprints/MyBP"`)
- `include_event_graph` (bool, 선택) — 이벤트 그래프 포함 (기본: true)
- `include_functions` (bool, 선택) — 함수 목록 포함 (기본: true)
- `include_variables` (bool, 선택) — 변수 목록 포함 (기본: true)
- `include_interfaces` (bool, 선택) — 인터페이스 포함 (기본: true)
- 응답: `{ blueprint_name, parent_class, variables, functions, event_graph, interfaces }`

`compile_blueprint`

블루프린트를 컴파일하여 GeneratedClass를 업데이트한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- 응답: `{ name, compiled: true }`

`delete_blueprint`

블루프린트 에셋을 프로젝트에서 삭제한다.

- 파라미터:
- `blueprint_path` (str, 필수) — 에셋 경로 (예: `"/Game/Blueprints/MyBP"`)
- `force` (bool, 선택) — 다른 에셋이 참조 중이어도 강제 삭제 (기본: false)
- 응답: `{ status: "success" }`

`spawn_blueprint_actor`

블루프린트의 인스턴스를 레벨에 스폰한다. 블루프린트는 사전에 컴파일되어 있어야 한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `name` (str, 필수) — 스폰할 액터의 고유 이름
- `location` (list[float], 선택) — `[x, y, z]` (기본: `[0, 0, 0]`)
- `rotation` (list[float], 선택) — `[pitch, yaw, roll]` (기본: `[0, 0, 0]`)
- `scale` (list[float], 선택) — `[x, y, z]` (기본: `[1, 1, 1]`)
- 응답: `{ status: "success" }`

Blueprint Component (3개)

CRD — 블루프린트 SCS 컴포넌트 조작 (Update는 Property 커맨드로 수행)

커맨드	CRUD	설명
`add_component_to_blueprint`	C	블루프린트에 컴포넌트 추가
`read_components_from_blueprint`	R	블루프린트의 컴포넌트 목록 조회
`remove_component_from_blueprint`	D	블루프린트에서 컴포넌트 제거

`add_component_to_blueprint`

블루프린트의 컴포넌트 계층(SCS)에 새 컴포넌트를 추가한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `component_class` (str, 필수) — 컴포넌트 클래스명
- `component_name` (str, 선택) — 컴포넌트 이름 (생략 시 자동)
- `parent_component` (str, 선택) — 부모 컴포넌트 이름 (생략 시 루트에 추가)
- 응답: `{ status: "success" }`

`read_components_from_blueprint`

블루프린트의 SCS 컴포넌트 계층을 반환한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- 응답: `{ blueprint_name, count, components: [{ name, class, is_root, parent }] }`

`remove_component_from_blueprint`

블루프린트에서 컴포넌트를 제거한다. 자식은 부모로 재배치된다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `component_name` (str, 필수) — 제거할 컴포넌트 이름
- 응답: `{ status: "success" }`

Property (2개)

U (Update only) — 블루프린트 컴포넌트 속성 설정

커맨드	CRUD	설명
`set_static_mesh_properties`	U	StaticMeshComponent에 메시/머티리얼 설정
`set_physics_properties`	U	PrimitiveComponent에 물리 속성 설정

`set_static_mesh_properties`

블루프린트 내 StaticMeshComponent에 스태틱 메시와 머티리얼을 설정한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `component_name` (str, 필수) — 대상 StaticMeshComponent 이름
- `static_mesh` (str, 선택) — 메시 에셋 경로 (예: `"/Engine/BasicShapes/Cube.Cube"`)
- `material` (str, 선택) — 머티리얼 에셋 경로
- 응답: `{ status: "success" }`

`set_physics_properties`

블루프린트 내 PrimitiveComponent의 물리 속성을 설정한다. 제공한 값만 변경된다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `component_name` (str, 필수) — 대상 PrimitiveComponent 이름
- `simulate_physics` (bool, 선택) — 물리 시뮬레이션 활성화
- `mass` (float, 선택) — 질량 (kg)
- `linear_damping` (float, 선택) — 선형 감쇠
- `angular_damping` (float, 선택) — 각도 감쇠
- 응답: `{ status: "success" }`

Material (6개)

RU — 머티리얼 조회 및 적용

커맨드	CRUD	설명
`get_available_materials`	R	프로젝트의 머티리얼 목록 조회
`get_actor_material_info`	R	액터의 머티리얼 슬롯 정보 조회
`get_blueprint_material_info`	R	블루프린트 컴포넌트의 머티리얼 슬롯 정보 조회
`apply_material_to_actor`	U	레벨 액터에 머티리얼 적용
`apply_material_to_blueprint`	U	블루프린트 컴포넌트에 머티리얼 적용
`set_mesh_material_color`	U	블루프린트 컴포넌트의 머티리얼 색상 변경

`get_available_materials`

프로젝트에서 사용 가능한 머티리얼 에셋 목록을 반환한다.

- 파라미터:
- `search_path` (str, 선택) — 검색 경로 (기본: `"/Game/"`)
- `include_engine_materials` (bool, 선택) — 엔진 내장 머티리얼 포함 (기본: true)
- 응답: `{ count, materials: [{ name, path, package, class }] }`

`get_actor_material_info`

액터의 메시 컴포넌트에 적용된 머티리얼 정보를 반환한다.

- 파라미터:
- `actor_name` (str, 필수) — 대상 액터 이름
- 응답: `{ actor_name, total_slots, material_slots: [{ slot_index, material_name, material_path, material_class }] }`

`get_blueprint_material_info`

블루프린트 내 StaticMeshComponent의 머티리얼 슬롯 정보를 반환한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `component_name` (str, 필수) — 대상 StaticMeshComponent 이름
- 응답: `{ blueprint_name, component_name, has_static_mesh, material_slots: [{ slot_index, material_name, material_path }] }`

`apply_material_to_actor`

레벨 내 액터의 StaticMeshComponent에 머티리얼을 적용한다.

- 파라미터:
- `actor_name` (str, 필수) — 대상 액터 이름
- `material_path` (str, 필수) — 머티리얼 에셋 경로
- `material_slot` (int, 선택) — 머티리얼 슬롯 인덱스 (기본: 0)
- 응답: `{ status: "success" }`

`apply_material_to_blueprint`

블루프린트 내 PrimitiveComponent에 머티리얼을 적용한다.

- 파라미터:

- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `component_name` (str, 필수) — 대상 PrimitiveComponent 이름
- `material_path` (str, 필수) — 머티리얼 에셋 경로
- `material_slot` (int, 선택) — 머티리얼 슬롯 인덱스 (기본: 0)
- 응답: `{ status: "success" }`

`set_mesh_material_color`

블루프린트 컴포넌트의 머티리얼에 색상 파라미터를 설정한다. Dynamic Material Instance를 생성하여 적용한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `component_name` (str, 필수) — 대상 PrimitiveComponent 이름
- `color` (list[float], 필수) — `[R, G, B]` 또는 `[R, G, B, A]` (0.0~1.0)
- `material_slot` (int, 선택) — 머티리얼 슬롯 인덱스 (기본: 0)
- `parameter_name` (str, 선택) — 셰이더 파라미터 이름 (기본: `"BaseColor"`)
- `material_path` (str, 선택) — 기본 머티리얼 경로
- 응답: `{ status: "success" }`

Static Mesh (2개)

R — 스태틱 메시 에셋 조회 (AssetRegistry)

커맨드	CRUD	설명
`get_static_meshes`	R	프로젝트의 스태틱 메시 목록 조회
`find_static_meshes_by_name`	R	이름으로 스태틱 메시 검색

`get_static_meshes`

프로젝트에서 사용 가능한 스태틱 메시 에셋 목록을 반환한다.

- 파라미터:
- `search_path` (str, 선택) — 검색 경로 (기본: `"/Game/"`)
- `include_engine_meshes` (bool, 선택) — 엔진 내장 메시 포함 (기본: true). `/Engine/BasicShapes/` 등
- 응답: `{ count, static_meshes: [{ name, path, package }], search_path }`

`find_static_meshes_by_name`

이름의 일부로 스태틱 메시를 검색한다.

- 파라미터:
- `search_text` (str, 필수) — 검색할 텍스트 (대소문자 구분, 부분 일치)
- `include_engine_meshes` (bool, 선택) — 엔진 내장 메시 포함 (기본: true)
- 응답: `{ count, static_meshes: [{ name, path, package }] }`

Blueprint Variable (2개)

CD — 블루프린트 멤버 변수 추가/제거

커맨드	CRUD	설명
`add_variable_to_blueprint`	C	블루프린트에 변수 추가
`remove_variable_from_blueprint`	D	블루프린트에서 변수 제거

`add_variable_to_blueprint`

블루프린트에 새 멤버 변수를 추가한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `variable_name` (str, 필수) — 변수 이름
- `variable_type` (str, 필수) — 변수 타입: `Bool`, `Int`, `Float`, `String`, `Name`, `Text`, `Vector`, `Rotator`, `Transform`
- `default_value` (str, 선택) — 기본값 (문자열)
- `is_instance_editable` (bool, 선택) — 인스턴스별 편집 허용 (기본: false)
- 응답: `{ status: "success", variable_name, variable_type }`

`remove_variable_from_blueprint`

블루프린트에서 멤버 변수를 제거한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `variable_name` (str, 필수) — 제거할 변수 이름

- 응답: `{ status: "success" }`

Blueprint Graph (7개)

CRUD — 블루프린트 이벤트 그래프의 노드와 연결 조작

커맨드	CRUD	설명
`read_blueprint_graph`	R	이벤트 그래프의 노드/핀/연결 상세 조회
`add_event_node`	C	이벤트 노드 추가 (BeginPlay, Tick 등)
`add_function_call_node`	C	함수 호출 노드 추가
`add_variable_get_node`	C	변수 Getter 노드 추가
`add_variable_set_node`	C	변수 Setter 노드 추가
`connect_nodes`	U	두 노드의 핀 연결
`disconnect_nodes`	D	핀의 모든 연결 해제

`read_blueprint_graph`

블루프린트의 이벤트 그래프를 상세 조회한다. 각 노드의 GUID, 클래스, 핀 목록, 연결 정보를 반환한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- 응답: `{ blueprint_name, graph_name, node_count, nodes: [{ node_id, class, title, position, pins }] }`

`add_event_node`

이벤트 그래프에 이벤트 노드를 추가한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `event_type` (str, 필수) — 이벤트 타입: `BeginPlay`, `Tick`, `EndPlay`, `ActorBeginOverlap`, `ActorEndOverlap`, `AnyDamage`, `Hit`
- `node_position` (list[int], 선택) — `[X, Y]` 그래프 위치 (기본: `[0, 0]`)
- 응답: `{ status: "success", node_id, pins: [...] }`

`add_function_call_node`

이벤트 그래프에 함수 호출 노드를 추가한다. 부모 클래스 → 공통 UE 라이브러리 순으로 함수를 자동 검색한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `function_name` (str, 필수) — 함수 이름 (예: `"PrintString"`, `"SetActorLocation"`, `"Delay"`)
- `target_class` (str, 선택) — 함수를 소유한 클래스 (예: `"KismetSystemLibrary"`). 생략 시 자동 검색
- `node_position` (list[int], 선택) — `[X, Y]` 그래프 위치 (기본: `[0, 0]`)
- 응답: `{ status: "success", node_id, function_name, pins: [...] }`

`add_variable_get_node`

이벤트 그래프에 변수 Getter 노드를 추가한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `variable_name` (str, 필수) — 블루프린트에 존재하는 변수 이름
- `node_position` (list[int], 선택) — `[X, Y]` 그래프 위치 (기본: `[0, 0]`)
- 응답: `{ status: "success", node_id, variable_name, pins: [...] }`

`add_variable_set_node`

이벤트 그래프에 변수 Setter 노드를 추가한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `variable_name` (str, 필수) — 블루프린트에 존재하는 변수 이름
- `node_position` (list[int], 선택) — `[X, Y]` 그래프 위치 (기본: `[0, 0]`)
- 응답: `{ status: "success", node_id, variable_name, pins: [...] }`

`connect_nodes`

이벤트 그래프에서 두 노드의 핀을 연결한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `source_node_id` (str, 필수) — 소스 노드의 GUID (`add_*_node` 응답의 `node_id`)
- `source_pin_name` (str, 필수) — 출력 핀 이름 (예: `"then"`, `"ReturnValue"`)
- `target_node_id` (str, 필수) — 타겟 노드의 GUID

- `target_pin_name` (str, 필수) — 입력 핀 이름 (예: `"execute"`, `"InString"`)
- 응답: `{ status: "success" }`
- 주요 핀 이름: `"execute"` (exec 입력), `"then"` (exec 출력), `"ReturnValue"` (반환값)

`disconnect_nodes`

핀의 모든 연결을 해제한다.

- 파라미터:
- `blueprint_name` (str, 필수) — 블루프린트 이름 또는 경로
- `node_id` (str, 필수) — 노드의 GUID
- `pin_name` (str, 필수) — 연결 해제할 핀 이름
- 응답: `{ status: "success", disconnected_count }`